

Documentation as Code

Why AI Has Made the Most Underrated Part of Engineering the Most Important One

By Sean Wong

There is a phrase that has echoed through software teams for so long it has become invisible:

"Agile killed documentation."

It is a comfortable line. It sounds like wisdom. It is repeated so often that many engineers stop questioning it. But the truth is more interesting, and more important.

Agile did not kill documentation. Agile killed documentation that nobody used.

The Great Misunderstanding

The Agile Manifesto famously says we value:

"Working software over comprehensive documentation."

Notice what it does *not* say. It does not say "working software instead of documentation." It does not say "no documentation." It does not say "documentation is worthless."

It says *over*.

That word matters. In 2001, this was a necessary correction. Teams were drowning in hundreds of pages of requirements, architectural diagrams, and process artifacts. Documentation had become the deliverable. Software was secondary. Updates were rare. By the time development started, the documents were already out of date.

Agile was never anti-documentation. It was anti-waste.

Keep the documentation that creates value.
Eliminate the documentation that does not.

For more than two decades, that interpretation worked remarkably well. Teams wrote less, shipped faster, and treated documentation as a conversation instead of a contract.

Then AI arrived.

And suddenly, documentation became one of the most valuable engineering assets in the stack again.

The Real Shift: Documentation Has a New Audience

Historically, documentation existed for humans. Future developers, new team members, architects, auditors, support engineers. It was written so that someone six months from now could understand what happened today.

AI changes that assumption.

Today, documentation often has a new primary consumer: **large language models**.

That shift is small in wording but enormous in consequence.

Traditional documentation was written for future humans, read occasionally, updated when remembered, and lived in wikis or Confluence. It was static. Passive. A reference.

AI-native documentation is written for present-day AI. It is machine-consumable. It is read constantly. It is updated continuously. It lives in source control. It is active. It is execution context.

Documentation is no longer just historical knowledge. It has become **operational knowledge**.

The Vibe Coding Paradox

The rise of AI-assisted development—often called *vibe coding*—has created a strange paradox.

Many developers assume that AI means we need *less* documentation. The machine can figure it out. We can just prompt and go.

The opposite is true.

The fundamental limitation exposed by AI-native development is simple:

Large language models have no organizational memory.

When you pair with a human engineer, context is shared. They understand architecture decisions, team conventions, domain terminology, previous trade-offs, and historical constraints.

An LLM understands none of that. Every conversation begins with a blank slate. Every prompt starts at zero. Every missing piece of context becomes an opportunity for hallucination.

An LLM is, in many ways, an **amnesiac genius**. It can generate code at extraordinary speed. But unless you provide rich context, it will confidently build the wrong thing.

This creates what I call **the Hallucination Tax**:

- More assumptions
- More architectural drift
- More code review overhead
- More rework

The problem is rarely model quality. The problem is **context quality**. And context has a familiar name: **documentation**.

Documentation Is Becoming Compiler Input

The biggest mistake teams make is treating documentation as a human artifact. In AI-native development, documentation behaves more like source code.

Consider the difference:

Weak prompt:

Build a login page.

Strong prompt:

Read SPEC.md and .cursorrules.

Build a login page.

Requirements:

- Use existing Button component
 - Use Zustand store
 - Follow shadcn/ui patterns
- Don't:
- Modify auth logic
 - Create new API endpoints

The second prompt generates dramatically better outcomes because documentation acts as **constraints**. The AI is not simply generating code—it is compiling instructions into implementation.

Documentation has effectively become **source code for AI behavior**.

Prompt-First Documentation: Three Principles

Once you accept that AI is a primary consumer of documentation, the form changes.

The objective is no longer:

"What should future developers know?"

Instead, it becomes:

"What must an AI understand before it writes code?"

This leads to three practical principles.

Principle 1: Extract, Don't Write

Documentation should be generated from implementation whenever possible. After completing a feature, documentation becomes a by-product of development rather than an additional burden.

Principle 2: Types Are Better Than Prose

The most powerful specification is not English. It is structure. A single type definition communicates allowed values, constraints, relationships, and boundaries more effectively than multiple paragraphs of prose. Humans understand it, compilers understand it, LLMs understand it. This is **executable documentation**.

Principle 3: Scope Ruthlessly

Ambiguity creates complexity. Every AI task should contain explicit boundaries. A strong **DON'T** section often eliminates entire categories of mistakes. The best prompts are often defined as much by what they prohibit as what they request.

The Three-Layer Specification Model

Successful AI-assisted teams naturally converge on three distinct specification layers.

Think of it as onboarding an exceptionally talented engineer who suffers complete memory

loss between conversations.

Layer 1: Application Spec — What Are We Building?

The product truth. Contains business goals, domain models, user workflows, business rules, and explicit non-goals. Purpose: **Prevent the AI from inventing features.**

Layer 2: Architecture Spec — How Should It Be Built?

Engineering standards, typically stored in `.cursorrules`, `CLAUDE.md`, `AGENTS.md`, or `PROJECT_CONTEXT.md`. Purpose: **Prevent architectural drift.**

Layer 3: Task Spec — What Needs To Be Built Right Now?

The implementation request. Combines Application context + Architecture constraints + immediate objective.

Documentation as a Learning System

The most powerful aspect is that documentation becomes **self-improving**.

When the AI violates a pattern, don't just fix the code—update the specification. The next prompt improves. And the next. Documentation evolves into a living system that continuously trains your AI collaborator.

Documentation Is Becoming a Programming Language

The bottleneck in software development has moved. For decades it was coding speed. Today it is **context quality**.

The teams that thrive in the AI era will not necessarily have the largest models. They will have clearer specifications, stronger architectural standards, reusable context, and better constraint systems.

Agile optimized human-to-human communication.

AI-native engineering optimizes human-to-LLM communication.

Documentation is no longer something you write *after* development. It actively *shapes* development. It constrains decisions. It defines boundaries. It guides implementation.

In many ways, it has become a new programming language—not one executed by machines directly, but one interpreted by AI systems on our behalf.

Agile didn't kill documentation.

AI has completed its transformation.

Documentation is no longer an artifact.

Documentation is becoming code.

— *Sean Wong* —